

Chapter 5: Databases and SQL



Outlines

- Basics
 - Queries
 - Subqueries
- Beyond basics
 - Query Optimization
 - NOSQL

Basics

What is a database?

- Database:
 - A collection of related data.
- Data:
 - Known facts that can be recorded and have an implicit meaning.
- Database Management System (DBMS):
 - A software package/system to facilitate the creation and maintenance of a computerized database.
- Database System:
 - The DBMS software together with the data itself. Sometimes, the application programs and interfaces are also included.

Data Definition Language (DDL)

- Specification notation for defining the database schema

Example: **create table** *instructor* (
 ID **char**(5),
 name **varchar**(20),
 dept_name **varchar**(20),
 salary **numeric**(8,2))

- DDL compiler generates a set of table templates stored in a *data dictionary*
- Data dictionary contains metadata (i.e., data about data)
 - Database schema
 - Integrity constraints
 - Primary key (ID uniquely identifies instructors)
 - Authorization
 - Who can access what

Data Manipulation Language (DML)

- Language for accessing and updating the data organized by the appropriate data model
 - DML also known as query language
- There are basically two types of data-manipulation language
 - **Procedural DML** -- require a user to specify what data are needed and how to get those data.
 - **Declarative DML** -- require a user to specify what data are needed without specifying how to get those data.
- Declarative DMLs are usually easier to learn and use than are procedural DMLs.
- Declarative DMLs are also referred to as non-procedural DMLs
- The portion of a DML that involves information retrieval is called a **query** language.

Structured Query Language

- SQL query takes as input several tables (possibly only one) and always returns a single table.

- Example to find all instructors in Comp. Sci. dept

select *name*

from *instructor*

where *dept_name* = 'Comp. Sci.'

- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL
 - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database

Create Table Construct

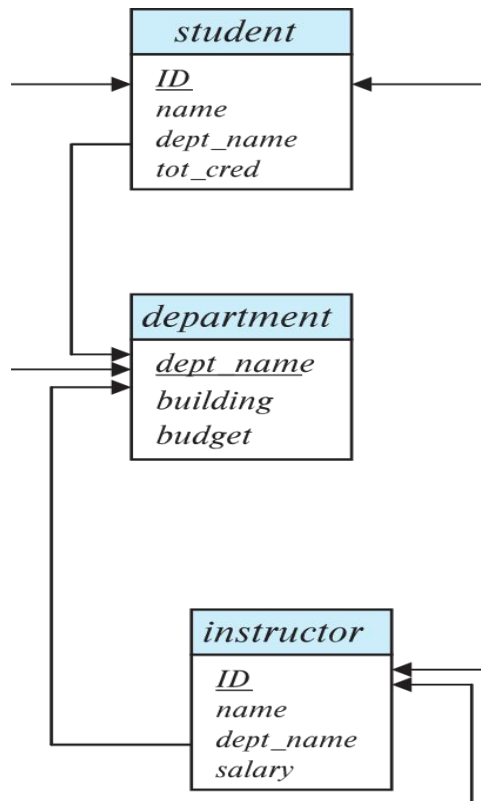
- An SQL relation is defined using the **create table** command:

create table *r*

$(A_1 D_1, A_2 D_2, \dots, A_n D_n,$
 $(\text{integrity-constraint}_1),$
 $\dots,$
 $(\text{integrity-constraint}_k))$

- r is the name of the relation
 - each A_i is an attribute name in the schema of relation r
 - D_i is the data type of values in the domain of attribute A_i
- Example:

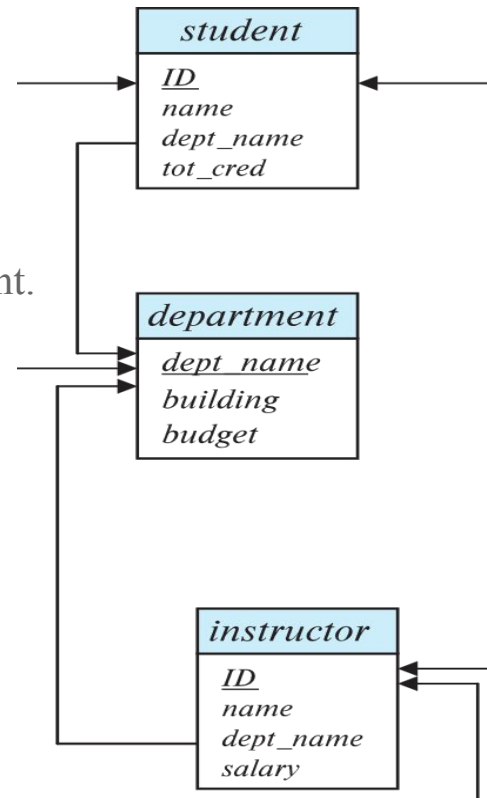
```
create table instructor (  
    ID          char(5),  
    name        varchar(20),  
    dept_name varchar(20),  
    salary     numeric(8,2))
```

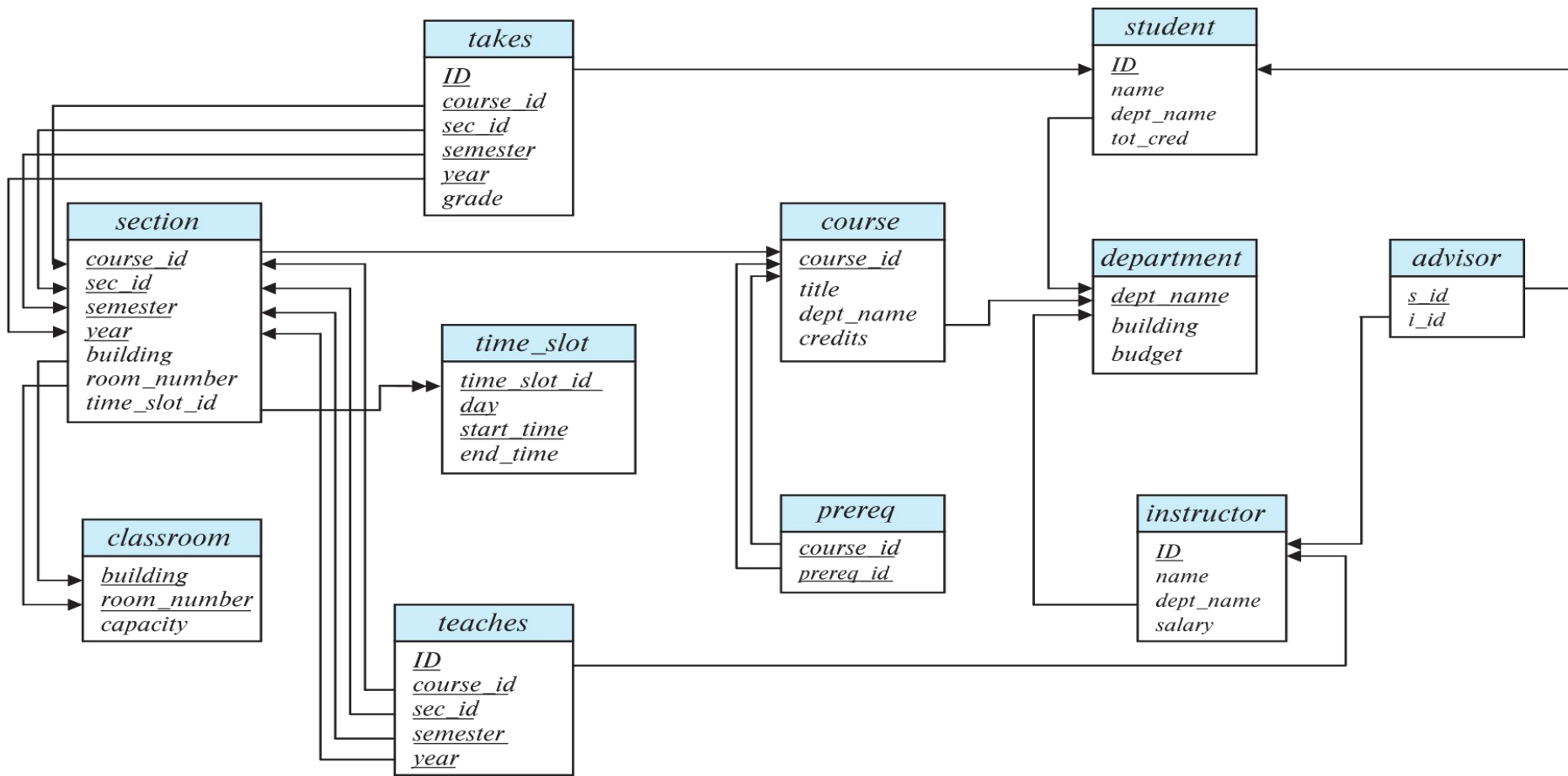


Integrity Constraints in Create Table

- Types of integrity constraints
 - primary key ($A_1, \dots, A_n$)
 - foreign key ($A_m, \dots, A_n$) references r
 - not null
- SQL prevents any update to the database that violates an integrity constraint.
- Example:

```
create table instructor (  
    ID          char(5),  
    name        varchar(20) not null,  
    dept_name   varchar(20),  
    salary      numeric(8,2),  
    primary key (ID),  
    foreign key (dept_name) references department);
```





And a Few More Relation Definitions

- **create table *student* (**
 ID **varchar(5),**
 name **varchar(20) not null,**
 dept_name **varchar(20),**
 tot_cred **numeric(3,0),**
 primary key (*ID*),
 foreign key (*dept_name*) references *department*);

- **create table *takes* (**
 ID **varchar(5),**
 course_id **varchar(8),**
 sec_id **varchar(8),**
 semester **varchar(6),**
 year **numeric(4,0),**
 grade **varchar(2),**
 primary key (*ID*, *course_id*, *sec_id*, *semester*, *year*) ,
 foreign key (*ID*) references *student*,
 foreign key (*course_id*, *sec_id*, *semester*, *year*) references *section*);

Modification of the Database

- Deletion of tuples from a given relation.
- Insertion of new tuples into a given relation
- Updating of values in some tuples in a given relation

Deletion

- Delete all instructors

delete from *instructor*

- Delete all instructors from the Finance department

delete from *instructor*

where *dept_name* = 'Finance';

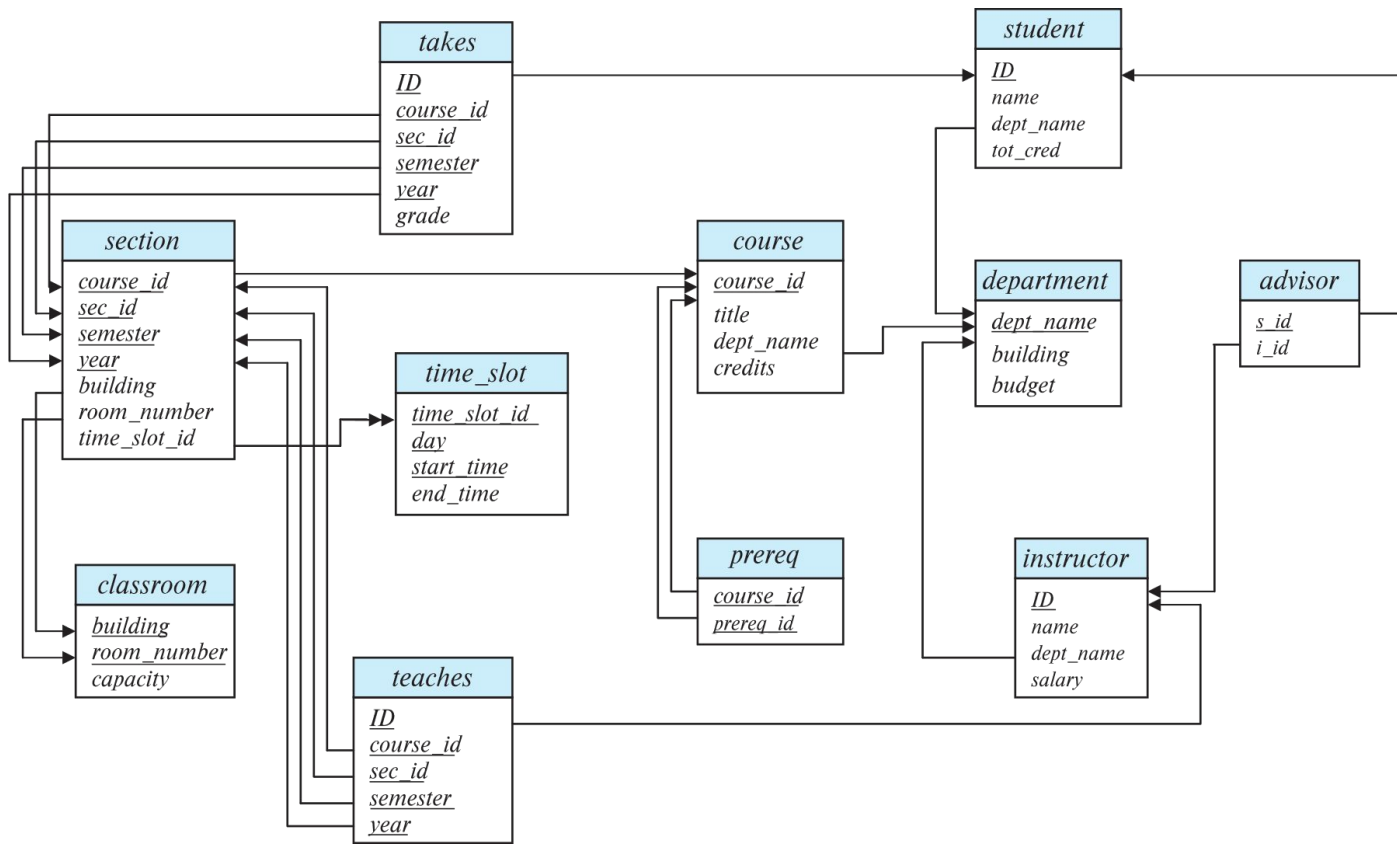
- *Delete all tuples in the instructor relation for those instructors associated with a department located in the Watson building.*

delete from *instructor*

where *dept_name* **in** (**select** *dept_name*

from *department*

where *building* = 'Watson');



Deletion (Cont.)

- Delete all instructors whose salary is less than the average salary of instructors

delete from *instructor*

where *salary* < (**select avg** (*salary*)

from *instructor*);

- Solution used in SQL:
 1. First, compute **avg** (*salary*) and find all tuples to delete
 2. Next, delete all tuples found above (without recomputing **avg** or retesting the tuples)

Insertion

- Add a new tuple to *course*

insert into *course*

values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- or equivalently

insert into *course* (*course_id*, *title*, *dept_name*, *credits*)

values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- Add a new tuple to *student* with *tot_creds* set to null

insert into *student*

values ('3003', 'Green', 'Finance', *null*);

Insertion (Cont.)

- Make each student in the Music department who has earned more than 144 credit hours an instructor in the Music department with a salary of \$18,000.

```
insert into instructor  
    select ID, name, dept_name, 18000  
from student  
where dept_name = 'Music' and total_cred > 144;
```

- The **select from where** statement is evaluated fully before any of its results are inserted into the relation.

Otherwise queries like

```
insert into table1 select * from table1
```

would cause problem

Updates

- Give a 5% salary raise to all instructors

```
update instructor  
  set salary = salary * 1.05
```

- Give a 5% salary raise to those instructors who earn less than 70000

```
update instructor  
  set salary = salary * 1.05  
  where salary < 70000;
```

- Give a 5% salary raise to instructors whose salary is less than average

```
update instructor  
  set salary = salary * 1.05  
  where salary < (select avg (salary) from instructor);
```

Updates (Cont.)

- Increase salaries of instructors whose salary is over \$100,000 by 3%, and all others by a 5%

- Write two **update** statements:

update *instructor*

set *salary* = *salary* * 1.03

where *salary* > 100000;

update *instructor*

set *salary* = *salary* * 1.05

where *salary* <= 100000;

- The order is important
- Can be done better using the **case** statement (next slide)

Basic Query Structure

- A typical SQL query has the form:

select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P

- A_i represents an attribute
 - R_i represents a relation
 - P is a predicate.
- The result of an SQL query is a relation.

The select Clause

- The **select** clause lists the attributes desired in the result of a query
 - corresponds to the projection operation of the relational algebra
- Example: find the names of all instructors:
select *name*
from *instructor*
- NOTE: SQL names are case insensitive (i.e., you may use upper- or lower-case letters.)
 - E.g., *Name* $\equiv$ *NAME* $\equiv$ *name*
 - Some people use upper case wherever we use bold font.

The select Clause (Cont.)

- SQL allows duplicates in relations as well as in query results.

```
select dept_name  
from instructor
```

- To force the elimination of duplicates, insert the keyword **distinct** after select.
- Find the department names of all instructors, and remove duplicates

```
select distinct dept_name  
from instructor
```

- The keyword **all** specifies explicitly that duplicates should not be removed.

```
select all dept_name  
from instructor
```

dept_name
Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

The select Clause (Cont.)

- An asterisk in the select clause denotes “all attributes”

```
select *  
from instructor
```

- An attribute can be a literal with no **from** clause

```
select '437'
```

- Results is a table with one column and a single row with value “437”
- Can give the column a name using:

```
select '437' as FOO
```

- An attribute can be a literal with **from** clause

```
select 'A'  
from instructor
```

- Result is a table with one column and N rows (number of tuples in the *instructors* table), each row with value “A”

The select Clause (Cont.)

- The **select** clause can contain arithmetic expressions involving the operation, +, −, *, and /, and operating on constants or attributes of tuples.

- The query:

```
select ID, name, salary/12  
from instructor
```

would return a relation that is the same as the *instructor* relation, except that the value of the attribute *salary* is divided by 12.

- Can rename “*salary/12*” using the **as** clause:

```
select ID, name, salary/12 as monthly_salary
```

The where Clause

- The **where** clause specifies conditions that the result must satisfy
 - Corresponds to the selection predicate of the relational algebra.

- To find all instructors in Comp. Sci. dept

```
select name
from instructor
where dept_name = 'Comp. Sci.'
```

- SQL allows the use of the logical connectives **and, or, and not**
- The operands of the logical connectives can be expressions involving the comparison operators **<, <=, >, >=, =, and <>**.
- Comparisons can be applied to results of arithmetic expressions
- To find all instructors in Comp. Sci. dept with salary > 70000

```
select name
from instructor
where dept_name = 'Comp. Sci.' and salary > 70000
```

<i>name</i>
Katz
Brandt

The from Clause

- The **from** clause lists the relations involved in the query
 - Corresponds to the Cartesian product operation of the relational algebra.
- Find the Cartesian product *instructor X teaches*

```
select *  
from instructor, teaches
```

 - generates every possible instructor – teaches pair, with all attributes from both relations.
 - For common attributes (e.g., *ID*), the attributes in the resulting table are renamed using the relation name (e.g., *instructor.ID*)
- Cartesian product not very useful directly, but useful combined with where-clause condition (selection operation in relational algebra).

Examples

- Find the names of all instructors who have taught some course
 - **select** *name, course_id*
from *instructor , teaches*
where *instructor.ID = teaches.ID*
- Find the names of all instructors in the Art department who have taught some course and the course_id
 - **select** *name, course_id*
from *instructor , teaches*
where *instructor.ID = teaches.ID*
and instructor. dept_name = 'Art'

<i>name</i>	<i>course_id</i>
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181

The Rename Operation

- The SQL allows renaming relations and attributes using the **as** clause:

old-name as new-name

- **as** clause can appear in both the **select** and **from** clauses.

- **select** *name, course_id*
from *instructor as I, teaches as T*
where *I.ID = T.ID*

- Find the names of all instructors who have a higher salary than some instructor in 'Comp. Sci'.

- **select distinct** *T.name*
from *instructor as T, instructor as S*
where *T.salary > S.salary and S.dept_name = 'Comp. Sci.'*

- Keyword **as** is optional and may be omitted

instructor as T $\equiv$ *instructor T*

Ordering the Display of Tuples

- List in alphabetic order the names of all instructors

```
select dept_name, name  
from   instructor  
order by name desc
```

- We may specify **desc** for descending order or **asc** for ascending order, for each attribute; ascending order is the default.

- Example: **order by name desc**

- Can sort on multiple attributes

- Example: **order by dept_name, name**

```
select dept_name, name  
from   instructor  
order by dept_name, name
```

Where Clause Predicates

- SQL includes a **between** comparison operator
- Example: Find the names of all instructors with salary between \$90,000 and \$100,000 (that is, $\geq$ \$90,000 and $\leq$ \$100,000)
 - **select** *name*
from *instructor*
where *salary* **between** 90000 **and** 100000
 - **select** *name*
from *instructor*
where *salary* $\leq$ 100000 **and** *salary* $\geq$ 90000
- Tuple comparison
 - **select** *name, course_id*
from *instructor, teaches*
where (*instructor.ID, dept_name*) = (*teaches.ID*, 'Biology');
 - **select** *name, course_id*
from *instructor, teaches*
where *instructor.ID* = *teaches.ID* **and** *dept_name* = 'Biology');

Aggregate Functions

- These functions operate on the multiset of values of a column of a relation, and return a value
 - avg:** average value
 - min:** minimum value
 - max:** maximum value
 - sum:** sum of values
 - count:** number of values

Aggregate Functions Examples

- Find the average salary of instructors in the Computer Science department
 - **select** **avg** (*salary*)
 from *instructor*
 where *dept_name*= 'Comp. Sci.';
- Find the total number of instructors who teach a course in the Spring 2018 semester
 - **select count** (**distinct** *ID*)
 from *teaches*
 where *semester* = 'Spring' **and** *year* = 2018;
- Find the number of tuples in the *course* relation
 - **select count** (*)
 from *course*;

Aggregate Functions – Group By

- Find the average salary of instructors in each department
 - `select dept_name, avg (salary) as avg_salary`
`from instructor`
`group by dept_name;`

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

<i>dept_name</i>	<i>avg_salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Aggregation (Cont.)

- Attributes in **select** clause outside of aggregate functions must appear in **group by** list

- /* erroneous query */

```
select dept_name, ID, avg (salary)
from instructor
group by dept_name;
```

Aggregate Functions – Having Clause

- Find the names and average salaries of all departments whose average salary is greater than 42000

```
select dept_name, avg (salary) as avg_salary
from instructor
group by dept_name
having avg (salary) > 42000;
```

- Note:
 - predicates in the **having** clause are applied **after** the formation of groups
 - predicates in the **where** clause are applied **before** forming groups

Nested Subqueries

- SQL provides a mechanism for the nesting of subqueries. A **subquery** is a **select-from-where** expression that is nested within another query.
- The nesting can be done in the following SQL query

```
select  $A_1, A_2, \dots, A_n$   
from  $r_1, r_2, \dots, r_m$   
where  $P$ 
```

as follows:

- **from clause:** r_i can be replaced by any valid subquery
- **where clause:** P can be replaced with an expression of the form:

B <operation> (subquery)

B is an attribute and <operation> to be defined later.

- **select clause:**
 A_i can be replaced by a subquery that generates a single value.

Test for Empty Relations

- The **exists** construct returns the value **true** if the argument subquery is nonempty.
 - **exists** $r \Leftrightarrow r \neq \emptyset$
 - **not exists** $r \Leftrightarrow r = \emptyset$

Use of “exists” Clause

- Yet another way of specifying the query “Find all courses taught in both the Fall 2017 semester and in the Spring 2018 semester”
- **exists** $r \Leftrightarrow r \neq \emptyset$
- **not exists** $r \Leftrightarrow r = \emptyset$

```
select course_id
from section as S
where semester = 'Fall' and year = 2017 and
      exists (select *
              from section as T
              where semester = 'Spring' and year = 2018
              and S.course_id = T.course_id);
```

- **Correlation name** – variable S in the outer query
- **Correlated subquery** – the inner query

Use of “not exists” Clause

- Find all students who have taken all courses offered in the Biology department.

```
select distinct S.ID, S.name  
from student as S  
where not exists ( (select course_id  
                    from course  
                    where dept_name = 'Biology')  
except  
                (select T.course_id  
                 from takes as T  
                 where S.ID = T.ID));
```

- First nested query lists all courses offered in Biology
 - Second nested query lists all courses a particular student took
-
- Note that $X - Y = \emptyset \Leftrightarrow X \subseteq Y$
 - Note: Cannot write this query using = all and its variants

Subqueries in the From Clause

- SQL allows a subquery expression to be used in the **from** clause
- Find the average instructors' salaries of those departments where the average salary is greater than \$42,000."

```
select dept_name, avg_salary
  from ( select dept_name, avg (salary) as avg_salary
         from instructor
         group by dept_name)
 where avg_salary > 42000;
```

- Note that we do not need to use the **having** clause
- Another way to write above query

```
select dept_name, avg_salary
  from ( select dept_name, avg (salary)
         from instructor
         group by dept_name)
  as dept_avg (dept_name, avg_salary)
 where avg_salary > 42000;
```

Scalar Subquery

- Scalar subquery is one which is used where a single value is expected
- List all departments along with the number of instructors in each department

```
select dept_name,  
       ( select count(*)  
         from instructor  
         where department.dept_name = instructor.dept_name)  
       as num_instructors  
from department;
```

- Runtime error if subquery returns more than one result tuple

Beyond basics

NOSQL

- NOSQL characteristics related to distributed databases and distributed systems.
 - NOSQL systems emphasize high availability, so replicating the data is inherent in many of these systems.
 - Scalability is another important characteristic, because many of the applications that use NOSQL systems tend to have data that keeps growing in volume.
 - High performance is another required characteristic.
- NOSQL characteristics related to data models and query languages.
 - Not Requiring a Schema: allowing semi-structured, self- describing data
 - Less Powerful Query Languages:
 - NOSQL systems typically provide a set of functions and operations (Create, Read, Update, and Delete operations)
 - Versioning: Some NOSQL systems provide storage of multiple versions of the data items, with the timestamps of when the data version was created.

NOSQL

NOSQL systems have been characterized into four major categories, with some additional categories that encompass other types of systems.

- Document-based NOSQL systems: These systems store data in the form of documents using well-known formats, such as JSON.
- NOSQL key-value stores: these systems have a simple data model based on fast access by the key to the value associated with the key
- Column-based or wide column NOSQL systems: these systems partition a table by column into column families, where each column family is stored in its own files.

NOSQL

NOSQL systems have been characterized into four major categories, with some additional categories that encompass other types of systems.

- Graph-based NOSQL systems: data is represented as graphs, and related nodes can be found by traversing the edges using path expressions.
- Hybrid NOSQL systems: These systems have characteristics from two or more of the above four categories.
- Object databases: OO databases store persistent objects permanently in secondary storage and allow the sharing of these objects among multiple programs and applications.
- XML databases: semistructured data

NOSQL

NOSQL system examples

- BigTable
 - Google's proprietary NOSQL system
 - Column-based or wide column store
- DynamoDB (Amazon)
 - Key-value data store
- Cassandra (Facebook)
 - Uses concepts from both key-value store and column-based systems
- MongoDB and CouchDB
 - Document stores
- Neo4J and GraphBase
 - Graph-based NOSQL systems
- OrientDB
 - Combines several concepts
- Database systems classified on the object model
 - Or native XML model